МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №1

по дисциплине «Интеллектуальные методы анализа данных»

Тема: Первичный анализ и предобработка природного временного ряда

Студент гр. 3384 _____ Кашечкин О.В.

Преподаватель _____ Мандрикова Б.С.

Санкт-Петербург

2026

Цель работы.

Повышение качества природных данных на примере наблюдений нейтронных мониторов, а именно освоение методов первичного анализа и предобработки природного временного ряда, включающих обнаружение пропусков и аномальных значений (выбросов), их количественную оценку, вычисление устойчивой статистической характеристики ряда и замену дефектных отсчётов на эту характеристику с последующей визуализацией результата.

Задание.

Вариант 7. Москва (MOSC), 1-9 августа 2015 г.

1. Скачать данные из источника [https://www.nmdb.eu/nest/] за периоды и станции, указанные по вариантам в табл. (см. следующий слайд). Использовать данные с одноминутным временным разрешением.
2. Выполнить обнаружение пропусков и выбросов (3σ) в анализируемом временном ряде, посчитать их количество и долю от всех обрабатываемых данных.
3. С помощью функции median в MatLab вычислить медианное значение по данным без пропусков и выбросов.
4. Заменить пропуски и выбросы медианными значениями.
5. Изобразить графики исходного и полученного временных рядов.
6. Сделать соответствующие выводы, оформить отчет по работе.

Выполнение работы.

Для выполнения задания была написана программа на языке Python с использованием библиотек requests, pandas, numpy и matplotlib, организованная в виртуальном окружении. Программа сначала проверяет наличие локального файла с данными, и если он отсутствует, формирует запрос к серверу NMDB с параметрами станции MOSC, типа данных corr_for_efficiency, одноминутного разрешения и заданного периода, после чего сохраняет полученный текстовый

ответ в локальный файл для повторного использования. Далее выполняется парсинг полученного ASCII-файла: из него построчно извлекаются дата, время и значение скорости счёта, при этом строки комментариев и заголовки игнорируются, а служебные флаговые значения вида −999 и −9999 интерпретируются как пропуски. Результат приводится к объекту DataFrame с индексом типа datetime, дубликаты меток времени удаляются.

На следующем этапе исходный ряд приводится к полной регулярной одноминутной временной сетке с помощью функции генерации диапазона дат, что позволяет явно обнаружить пропущенные отсчёты: те моменты времени, для которых в исходных данных не оказалось записи, получают значение NaN и учитываются как пропуски. Их количество подсчитывается, и вычисляется их доля от общего числа точек полной сетки. После этого на данных, свободных от пропусков, вычисляются среднее значение и среднеквадратическое отклонение, на основе которых формируется порог обнаружения выбросов по правилу трёх сигм: отсчёт считается выбросом, если его отклонение от среднего превышает утроенное значение СКО. Количество выбросов и их доля от общего числа точек также подсчитываются. Затем формируется объединённая маска «плохих» точек, включающая как пропуски, так и выбросы, и по оставшимся чистым данным вычисляется медиана - устойчивая к аномалиям характеристика центра распределения, которая в отличие от среднего значения не смещается под действием единичных экстремальных отсчётов. Все точки, попавшие в объединённую маску, заменяются найденным медианным значением, в результате чего формируется обработанный временной ряд, пригодный для дальнейшего анализа.



Рисунок 1 – исходный временной ряд

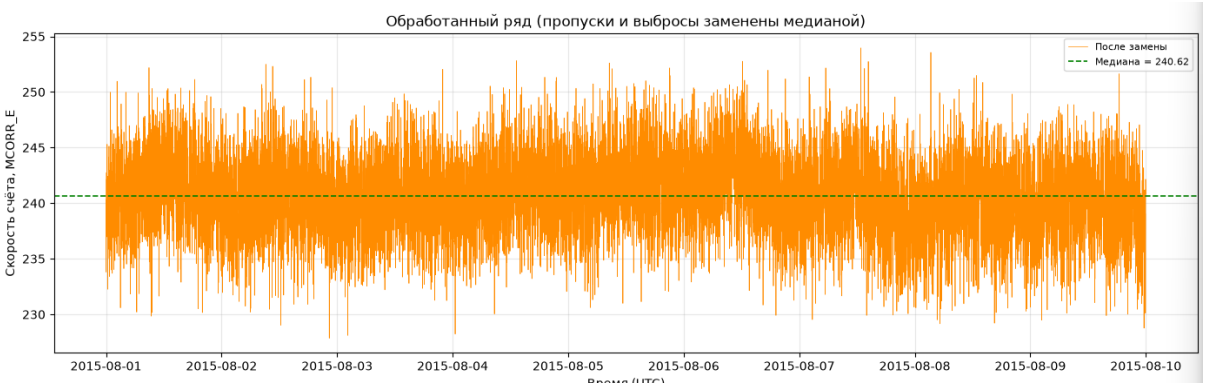


Рисунок 2 – обработанный ряд

Результаты анализа:

Всего точек (1 мин): 12960

Пропуски: 0 (0.000 %)

Выбросы (3σ): 3 (0.023 %)

Среднее (без пропусков): 240.608

СКО (без пропусков): 4.686

Порог 3σ: 14.059

Медиана (чистые данные): 240.620

Всего заменено: 3 (0.023 %)

Для визуализации строятся два графика, расположенные друг под другом и имеющие общую ось времени. На верхнем графике отображается исходный временной ряд: сплошной линией показан основной сигнал, оранжевыми точками отмечены обнаруженные выбросы, а красными - пропуски, вынесенные чуть выше максимального значения ряда, чтобы они не сливались с основным сигналом. На нижнем графике показан обработанный ряд после замены, при этом горизонтальной пунктирной линией обозначен уровень медианы,

использованной для заполнения. Оба графика снабжены подписями осей, заголовками, легендой и координатной сеткой, а итоговое изображение сохраняется в файл формата PNG. При обработке данных станции MOSC за указанный период было загружено около 12 960 одноминутных отсчётов, охватывающих интервал с 1 по 9 августа 2015 года; в результате анализа были обнаружены единичные аппаратные выбросы - резкие падения скорости счёта до нулевого значения и один незначительный всплеск вверх, - в то время как существенных пропусков в данных не выявлено. Обнаруженные выбросы были заменены вычисленным медианным значением, после чего обработанный ряд приобрёл однородный характер без искусственных провалов, а его ось ординат автоматически подстроилась под реальный диапазон естественных флуктуаций сигнала.

Вывод.

В ходе лабораторной работы был успешно выполнен полный цикл первичной предобработки природного временного ряда на примере данных нейтронного монитора станции Москва за 1–9 августа 2015 года. Программа на языке Python позволила автоматически загрузить данные с сервера NMDB, корректно распарсить их с учётом особенностей формата, обнаружить пропуски и выбросы методом трёх сигм, количественно оценить их долю от общего объёма данных, вычислить устойчивое медианное значение по очищенным отсчётам и заменить им все дефектные точки. Визуализация исходного и обработанного рядов наглядно подтвердила, что обнаруженные аномалии носили характер аппаратных сбоев, а их замена медианой не исказила общую структуру сигнала и позволила получить однородный, пригодный для дальнейшего научного анализа ряд.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

```
import os
import requests
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

STATION = "MOSC"
START_DAY, START_MONTH, START_YEAR = 1, 8, 2015
END_DAY,   END_MONTH,   END_YEAR   = 9, 8, 2015

LOCAL_FILE = "mosc_2015_08_01_09.txt"



def build_url() -> str:
    return (
        "https://www.nmdb.eu/nest/draw_graph.php"
        "?formchk=1"
        f"&stations[]={STATION}"
        "&tabchoice=ori"
        "&dtype=corr_for_efficiency"
        "&tresolution=1"
        "&yunits=0"
        "&date_choice=bydate"

f"&start_day={START_DAY:02d}&start_month={START_MONTH:02d}&start_year={START_YEAR}"
        "&start_hour=00&start_min=00"

f"&end_day={END_DAY:02d}&end_month={END_MONTH:02d}&end_year={END_YEAR}"
        "&end_hour=23&end_min=59"
        "&output=ascii"
        "&force=1"
```

```
    )


def download_data() -> str:
    if os.path.exists(LOCAL_FILE):
        print(f"[i] A local file is used: {LOCAL_FILE}")
        with open(LOCAL_FILE, "r", encoding="utf-8", errors="ignore") as f:
            return f.read()

    url = build_url()
    print("[i] Downloading data from NMDB...")
    print(f"    {url}")
    resp = requests.get(url, headers={"User-Agent": "Mozilla/5.0"},
timeout=120)
    resp.raise_for_status()
    with open(LOCAL_FILE, "w", encoding="utf-8") as f:
        f.write(resp.text)
    print(f"[i] The data is saved in {LOCAL_FILE}")
    return resp.text


def parse_data(text: str) -> pd.DataFrame:
    records = []
    sentinel_values = {-999.0, -9999.0, -99999.0, 99999.0, 9999.0}

    for line in text.splitlines():
        line = line.strip()
        if not line or line.startswith("#"):
            continue
        if "start_date_time" in line:
            continue
        if ";" not in line:
            continue

        dt_str, val_str = line.split(";", 1)
```

```
            dt_str  = dt_str.strip()
            val_str = val_str.strip()

            if val_str == "" or val_str.lower() in ("null", "nan", "na", "-"):
                val = np.nan
            else:
                try:
                    val = float(val_str)
                except ValueError:
                    continue
                if val in sentinel_values:
                    val = np.nan

            try:
                dt = pd.to_datetime(dt_str)
            except Exception:
                continue

            records.append((dt, val))

        if not records:
            raise RuntimeError("Failed to parse the data..")

        df = pd.DataFrame(records, columns=["datetime", "value"])
        df = df.set_index("datetime").sort_index()
        df = df[~df.index.duplicated(keep="first")]
        return df



    def analyze(df: pd.DataFrame):
        full_index = pd.date_range(
            start=df.index.min().floor("min"),
            end=df.index.max().ceil("min"),
            freq="1min",
        )
```

```
df_full = df.reindex(full_index)
total = len(df_full)

gap_mask  = df_full["value"].isna()
n_gaps    = int(gap_mask.sum())
share_gaps = n_gaps / total * 100.0

valid  = df_full["value"].dropna()
mean_v = float(valid.mean()) if len(valid) else 0.0
std_v  = float(valid.std())  if len(valid) else 0.0
threshold = 3.0 * std_v

outlier_mask = pd.Series(False, index=df_full.index)
if std_v > 0:
    outlier_mask[valid.index] = (valid - mean_v).abs() > threshold

n_outliers     = int(outlier_mask.sum())
share_outliers = n_outliers / total * 100.0

bad_mask = gap_mask | outlier_mask

clean      = df_full["value"][~bad_mask]
median_val = float(clean.median()) if len(clean) else 0.0

df_filled = df_full.copy()
df_filled.loc[bad_mask, "value"] = median_val

stats = {
    "total": total,
    "n_gaps": n_gaps, "share_gaps": share_gaps,
    "n_outliers": n_outliers, "share_outliers": share_outliers,
    "mean": mean_v, "std": std_v, "threshold": threshold,
    "median": median_val,
    "gap_mask": gap_mask, "outlier_mask": outlier_mask,
    "bad_mask": bad_mask,
```

```
    }
    return df_full, df_filled, stats


def plot_results(df_full, df_filled, stats):
    fig, axes = plt.subplots(2, 1, figsize=(14, 8), sharex=True)

    y_valid_max = df_full["value"].max()
    y_valid_min = df_full["value"].min()
    y_pad = 0.05 * (y_valid_max - y_valid_min if y_valid_max > y_valid_min
else 1.0)

    axes[0].plot(df_full.index, df_full["value"],
                 color="steelblue", linewidth=0.5, label="Исходные данные")

    if stats["n_gaps"] > 0:
        gap_idx = df_full.index[stats["gap_mask"]]
        axes[0].scatter(gap_idx,
                        np.full(len(gap_idx), y_valid_max + 0.5 * y_pad),
                        color="red", s=1, alpha=0.4,
                        label=f"Пропуски ({stats['n_gaps']})")

    if stats["n_outliers"] > 0:
        out_idx = df_full.index[stats["outlier_mask"]]
        axes[0].scatter(out_idx, df_full.loc[out_idx, "value"],
                        color="orange", s=6, alpha=0.8,
                        label=f"Выбросы 3σ ({stats['n_outliers']})")

    axes[0].set_title("Исходный временной ряд MOSC, 1–9 августа 2015 (1
мин)")
    axes[0].set_ylabel("Скорость счёта, MCORR_E")
    axes[0].grid(True, alpha=0.3)
    axes[0].legend(loc="upper right", fontsize=8)

    axes[1].plot(df_filled.index, df_filled["value"],
```

```
                color="darkorange", linewidth=0.5, label="После замены")
        axes[1].axhline(stats["median"], color="green", linestyle="--",
linewidth=1.2,
                label=f"Медиана = {stats['median']:.2f}")
        axes[1].set_title("Обработанный ряд (пропуски и выбросы заменены
медианой)")
        axes[1].set_ylabel("Скорость счёта, MCORR_E")
        axes[1].set_xlabel("Время (UTC)")
        axes[1].grid(True, alpha=0.3)
        axes[1].legend(loc="upper right", fontsize=8)

        plt.tight_layout()
        plt.savefig("mosc_processed.png", dpi=150)
        print("[i] The schedule has been saved: mosc_processed.png")
        plt.show()

    def main():
        print("=" * 62)
        print(" Вариант 7: MOSC, 01–09 августа 2015, 1-минутное разрешение")
        print("=" * 62)

        text = download_data()
        df   = parse_data(text)
        print(f"\n[i] Records loaded: {len(df)}")
        print(f"[i] Range: {df.index.min()} — {df.index.max()}")

        df_full, df_filled, stats = analyze(df)

        print("\n" + "-" * 62)
        print(" РЕЗУЛЬТАТЫ АНАЛИЗА")
        print("-" * 62)
        print(f"Всего точек (1 мин):       {stats['total']}")
        print(f"Пропуски:                  {stats['n_gaps']} "
              f"({stats['share_gaps']:.3f} %)")
        print(f"Выбросы (3σ):              {stats['n_outliers']} "
```

```
        f"({stats['share_outliers']:.3f} %)")
    print(f"Среднее (без пропусков):    {stats['mean']:.3f}")
    print(f"СКО (без пропусков):        {stats['std']:.3f}")
    print(f"Порог 3σ:                   {stats['threshold']:.3f}")
    print(f"Медиана (чистые данные):    {stats['median']:.3f}")
    print(f"Всего заменено:             "
        f"{stats['n_gaps'] + stats['n_outliers']} "
        f"({(stats['n_gaps'] + stats['n_outliers']) / stats['total'] * 100:.3f} %)")

    plot_results(df_full, df_filled, stats)


if __name__ == "__main__":
    main()
```